`select_dtypes`

Status: extends Practical 6 **Where it is used:** `backend/analytics.py` (`numeric_columns`, `describe_numeric`, `group_means`, `strongest_separators`, `profile`, `save_charts`), `scripts/profile_dataset.py` (`numeric_columns`, `profile`, `print_detail`) **Closest lab concept:** Practical 6, `df.columns` — `select_dtypes` is `df.columns` filtered by dtype, so the result you want is `.columns` on it

---

1. What it is

`df.select_dtypes(...)` takes a DataFrame and returns **a new DataFrame containing only the columns whose dtype matches**. It does not filter rows, it does not return a list, and it does not modify `df`.

```
df.select_dtypes(include="number")
```

returns a DataFrame with every `int64` / `float64` column and nothing else. `df.select_dtypes(include="object")` returns the text columns. `exclude=` is the mirror image: everything *except* the dtype you name.

You already know the result is shaped like a DataFrame, because Practical 6 taught `df.columns` and you can read a dtype list off it. The real `df.dtypes` of this project's `parkinsons.csv`, trimmed to the top and bottom of the list:

```
patient             str
fo_hz           float64
fhi_hz           float64
...
status           int64
...
ppe             float64
dtype: object
```

`select_dtypes(include="number")` is the same list with a filter on it. Practical 6 never needed it, because every lab dataset was numeric or had a hand-written column list. The moment a dataset mixes text and numbers — which every clinical file here does — picking the numbers by hand stops being reasonable, and `select_dtypes` does it by dtype rule instead.

---

2. Why the project needed it

Meridian profiles four UCI clinical datasets with different schemas: `heart_disease.csv` has 14 columns, `parkinsons.csv` has 24, `pima_diabetes.csv` has 9. Spec section 7.2 wants `describe()` on the numeric columns, a `groupby(target).mean()` over them, and a ranking of which numeric column separates the target classes best (`analytics.strongest_separators`).

None of that can be written once, because there is no shared column list. The datasets do not agree on names, on order, or on which columns exist. So `analytics.py` asks each frame what its numeric columns *are* rather than hard-coding a list per dataset. That question has exactly one answer: `select_dtypes(include="number")`.

This is also why the profiler (`scripts/profile_dataset.py`) can score twenty candidate files against one criterion, `MIN_NUMERIC = 5`, without knowing anything about any of them.

---

3. The four pieces actually used

3.1 It returns a DataFrame, so `.columns` comes next

The one call site in the whole project, `analytics.py:83`:

```
    cols = list(df.select_dtypes(include="number").columns)
```

Read it left to right: select the numeric columns, take their `.columns`, turn that into a Python `list`. The result is a plain list of strings — `['age', 'trestbps', 'chol', 'thalach', 'oldpeak', ...]`.

Note that `.columns` is doing real work here. `select_dtypes` alone would hand back a whole DataFrame, which is the wrong shape for everything this project does next: looping over names, slicing to a limit, removing one column, and passing names to `groupby`. `list(...)` also drops the pandas `Index` wrapper, so the value can be compared, sliced and stored without pandas types leaking into JSON later.

3.2 The two-step idiom: names first, then index

Every consumer of `numeric_columns()` wants **names**, not a frame. That is why the project uses the two-step form — `select_dtypes(...).columns` — rather than holding on to the frame. Once you have the names, four things become possible that a DataFrame would have blocked:

1. **Exclude the target.** A list comprehension over names can drop one (section 3.3).
2. **Slice to a limit.** `numeric_columns(df)[:8]` is list slicing, not frame slicing — `analytics.group_means` and `profile` both do it.
3. **Loop over names.** `for col in cols:` drives `strongest_separators` and the `group_means` chart.
4. **Re-index the frame by name.** `df[cols]` selects exactly those columns and returns a DataFrame again.

Step 4 is the bridge back to Practical 6. `df[cols]` where `cols` is a list of column names is the same operation as `.loc`, which you used:

```
df.loc[:, ["age", "chol"]]
df[["age", "chol"]]
```

The first line is the Practical 6 form. The second is what `analytics.py` writes — same result, no axis to state because a bare list can only ever mean columns. The short form is the idiomatic one; `.loc` is the explicit form. Both return the same frame. `analytics.py` uses the short form throughout — `df[cols].describe()` (line 157), `df.groupby(target)[cols].mean()` (line 187), `df[cols].mean()` (line 222).

3.3 `exclude_target` — the target is not a feature

`analytics.py:80-88`, quoted in full:

```
def numeric_columns(df, exclude_target=True):
    """Return numeric column names, optionally excluding the target."""
    target = find_target(df)
    cols = list(df.select_dtypes(include="number").columns)

    if exclude_target and target:
        cols = [c for c in cols if c != target]

    return cols
```

Read the last three lines as boolean filtering, which is Practical 6: `c != target` builds a True/False per column name and the comprehension keeps the Trues. It is the `c not in [...]` shape rather than the `&`/`|` mask you used, because these are strings in a list, not boolean columns in a frame — there is no index to align against.

Why the exclusion exists is stated in the comment at `analytics.py:218-219`:

```
    # The target itself is excluded. Grouping by target and then ranking the
    # target column always wins, which tells us nothing.
    cols = numeric_columns(df, exclude_target=True)
```

`find_target(df)` finds `status`, `target`, `Outcome` or `malignancy` by name (line 72-77). Those columns are numeric too — `status` is 0/1 — so `select_dtypes` cannot exclude them. It has no idea which column is the answer you are trying to predict. Dropping a column named in `TARGET_NAMES` is a job the dtype rule

cannot do, so it is done by hand afterwards.

The flag exists because `profile()` (line 267) still wants the target counted among the numeric columns it reports, while `strongest_separators` (line 220) must not have it. Same helper, two answers, one keyword argument.

3.4 The categorical side is a list comprehension, not `include="object"`

`profile_dataset.py:55-57` needs the other half — the non-numeric columns — and does **not** call `select_dtypes(include="object")`:

```
    numeric_cols = numeric_columns(df)
    categorical_cols = [c for c in df.columns
                        if c not in numeric_cols and c != find_target(df)]
```

This is a deliberate difference from `analytics.py`. It walks `df.columns` and keeps whatever is not numeric, which is `object` selection by subtraction. The reason is section 4. The same pattern appears again in `print_detail` (lines 110-111):

```
        categorical = [c for c in df.columns
                       if c not in numeric and c != target]
```

Worth noticing the second condition as well: the target is excluded from the categorical list too, exactly as it is from the numeric list. One column, two exclusion rules, so it never appears in either count.

---

4. Real bug: the `Pandas4Warning` on object selection

This project ran on **pandas 3.0.5**, and pandas 3 changed what dtype a text column has. The first version of the categorical-detection line passed the dtype **positionally**:

```
df.select_dtypes("object")
```

That is legal but fragile — the first positional parameter is `include`, so it happens to work, and it worked on pandas 2. On pandas 3 it emits:

```
Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtyp
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for
```

What the warning is actually about: in pandas 3 a column of strings is stored with the new `str` dtype, not `object`. Asking for `object` therefore matches nothing, so for backward compatibility pandas 3 also pulls `str` columns into the result and tells you it is doing so. That is the "deprecated behaviour" in the text — the behaviour will go away in a future release, and code depending on it will silently start returning empty frames.

What the fix turned out to be was not what anyone expected. Making the argument explicit is the obvious first move, and the code that actually ships is:

```
df.select_dtypes(include="number")
```

One honest correction to the story usually told here. The obvious fix — spelling it `df.select_dtypes(include="object")` — is *not* sufficient on pandas 3.0.5. Measured on this project's environment:

```
dtypes: {'a': dtype('int64'), 'b': <StringDtype(na_value=nan)>}
positional   -> cols ['b'] | warnings: ['Pandas4Warning']
include=object -> cols ['b'] | warnings: ['Pandas4Warning']
include=str  -> cols ['b'] | warnings: []
include=number -> cols ['a'] | warnings: []
exclude=str  -> cols ['a'] | warnings: []
```

Both the positional form and `include="object"` warn. The only forms that are silent are `include="str"` and `exclude="str"`.

So the code that ships does not select object columns at all. It uses `include="number"` (never warns, because the `str` ambiguity does not apply to numeric dtypes) and derives the categorical side by subtraction in `profile_dataset.py:56-57`. That is the version you can point at, and it is why no `Pandas4Warning` appears anywhere in the project today.

The transferable lesson is not "type the keyword". It is: **a deprecation warning is a prompt to re-check the dtype, not to change one string.** `df.dtypes` is the tool for that, and it is Practical 6.

---

5. When nothing matches, `select_dtypes` returns an empty frame

This is the failure mode the profiler actually caught, and it is worth knowing because it is silent.

`select_dtypes` does not raise when it finds no columns of the requested type. It returns a DataFrame with the right number of rows and zero columns. So `.columns` gives `[]`, `numeric_columns()` returns `[]`, and every downstream statistic becomes an empty dict instead of an error. `analytics.py` guards for it explicitly in two places — `describe_numeric` (lines 152-155):

```
    cols = numeric_columns(df)

    if not cols:
        return {}

    return to_native(df[cols].describe().to_dict())
```

and `group_means` (lines 182-185):

```
    cols = numeric_columns(df)[:limit]

    if not cols:
        return {}

    return to_native(df.groupby(target)[cols].mean().to_dict())
```

Without those guards, `df[cols].mean()` on an empty column list returns an empty Series and the API would report "no statistics" for a dataset that is actually fine — the same empty response as a genuine zero-row frame, with nothing to tell them apart. The `if not cols` guard is what makes "this dataset has no numeric columns" a distinguishable answer.

`save_charts` (line 413) carries the same guard into the plotting code:

```
    focus = separators[0]["column"] if separators else (cols[0] if cols else None)
```

A `None` focus means no histogram is drawn, rather than `cols[0]` raising `IndexError` on an empty list.

Where this actually bit the project — a file whose *every* column was read as text — is documented in `docs/concepts/pandas-missing-values.md`, section 4.

---

6. Questions this should answer

- What does `df.select_dtypes(include="number")` return, and how is that different from `df[df.dtypes == "float64"]`?
- Why does `analytics.numeric_columns` call `.columns` and wrap it in `list()` instead of returning the frame from `select_dtypes`?
- `df[cols]` where `cols` is a list of names — which Practical 6 method is this equivalent to, and why is `.iloc` the wrong one here?
- Why must the target column be excluded by name after `select_dtypes`, when `select_dtypes`

already knows which columns are numeric?

- What does `df.select_dtypes(include="number")` return when no column is numeric, and which two guards in `analytics.py` exist for it?
- On pandas 3, why does `select_dtypes(include="object")` warn, and what is the difference between that and `include="str"`?